

SENTIMENT ANALYSIS USING LSTM MODELS

A PROJECT REPORT

Submitted to

Laxman Devram Sonawane College of Arts and Commerce, Kalyan(west), 421301

*In partial fulfillment for the award of the
degree of*

**Master of Science in
Information Technology**



(Affiliated to University of Mumbai)

Submitted by

TEJAS ANIL KAPADE

Seat Number: 1313198

UNDER THE GUIDENCE OF

Dr. NRUPURA DIXIT

Prof. SABINA ANSARI

DEPARTMENT OF INFORMATION TECHNOLOGY

Kalyan, Maharashtra

Academic Year 2024-2025



The Kalyan Wholesale Merchants' Education Society's,
LAXMAN DEVRAM SONAWANE DEGREE COLLEGE

Department of Information Technology

M.Sc. Part - II

Certificate

This is to certify that the project entitled “**Sentiment Analysis Using LSTM Models**” of “**Mr. Tejas Anil Kapade, Roll No: 16, Seat_No: 1313198**” submitted to the Laxman Devram Sonwane College Affiliated to University of Mumbai in partial fulfillment of the requirement for the award of the degree of “**Postgraduate**” in “**Master of Science in Information Technology**” during the academic year 2024- 2025.

Project In-charge

Co-Ordinator In-Charge

External Examiner

College Seal

DECLARATION

I, **Tejas Anil Kapade** the student of Information Technology, declare that the work entitled “**Sentiment Analysis Using LSTM Models**” has been successfully completed under the guidance of

DR. NRUPURA DIXIT this dissertation work is submitted in partial fulfilment of the requirements for the award of Degree of Master of

Science in Information Technology during the academic year 2024-2025.

Further the matter embodied in the project report has not been submitted previously by anybody for the award of any degree or diploma to any university.

Student Feedback on Research Project (To be filled by Students after Project completion)

Student Name : Tejas Anil Kapade

Seat No. /Roll No. : 1313198

Email : me.tejaskapade@gmail.com

Department : Master of Science Information Technology (Artificial Intelligence)
Name of the Mentor : Ms. Sabina Ansari & Dr. Nrupura Dixit

Title of Research Project : Sentiment Analysis Using LSTM Models

Brief description of Project work carried out : To Detect Sentiment of Input is Positive or Negative

Year of completion of Research Project : 2024-25

Was your project work experience related to your major area of study?

- Yes, to a large degree
- Yes, to a slight degree
- No, not related at all

Indicate the degree to which you agree or disagree with the following statements

This experience has:	Strongly Agree	Agree	No opinion	Disagree	Strongly Disagree
Given me the opportunity to explore a career field					
Allowed me to apply classroom theory to practice					
Helped me develop my decision-making and problem-solving skills					
Expanded my knowledge about the work world before permanent employment					
Helped me develop my written and oral communication skills					

Provided a chance to use leadership skills (influence others, develop ideas with others, stimulate decision-making and action)					
Expanded my sensitivity to the ethical implications of the work involved					
Made it possible for me to be more confident in new situations					
Given me a chance to improve my interpersonal skills					
Helped me learn to handle responsibility and use my time wisely					
Helped me discover new aspects of myself that I didn't know existed before					
Helped me develop new interests and abilities					
Helped me clarify my career goals					
Allowed me to acquire information and/ or use equipment not available at my Institute					
Allowed me to realize socioeconomic/environmental issues.					

PROJECT SYNOPSIS

SENTIMENT ANALYSIS USING LSTM MODELS

Introduction

Sentiment analysis is an important application of Natural Language Processing (NLP) that focuses on identifying and classifying opinions expressed in textual data. With the rapid growth of online platforms, customers frequently share their experiences and feedback in the form of reviews. In the hospitality industry, hotel reviews play a crucial role in influencing customer decisions and improving service quality.

Manual analysis of a large volume of hotel reviews is time-consuming and inefficient. Therefore, an automated sentiment analysis system is required to extract meaningful insights from customer feedback. Deep learning techniques, especially Long Short-Term Memory (LSTM) networks, have shown promising results in handling sequential text data. LSTM models are capable of capturing contextual and long-term dependencies within text, making them suitable for sentiment classification tasks.

This project proposes an LSTM-based sentiment analysis system to classify hotel reviews as positive or negative, helping hotels and customers better understand overall customer sentiment.

Problem Statement

With the increasing number of hotel reviews available on online platforms, it has become difficult to manually analyze customer opinions and determine overall sentiment accurately. Traditional machine learning approaches often fail to capture contextual meaning and long-term dependencies in text data. Hence, there is a need for an automated and efficient sentiment analysis system using deep learning techniques to accurately classify hotel reviews into positive or negative categories.

Objectives of the Project

The main objectives of this project are:

- To analyze hotel reviews using Natural Language Processing techniques
- To design and implement an LSTM-based sentiment analysis model

- To classify hotel reviews as positive or negative
- To improve sentiment classification accuracy compared to traditional methods
- To demonstrate the application of deep learning in real-world text analysis

Scope of the Project

The scope of this project includes:

- Analysis of English-language hotel reviews only
- Binary sentiment classification (positive or negative)
- Use of LSTM neural networks for sentiment prediction

The project does not include multilingual sentiment analysis or neutral sentiment classification. However, the proposed system can be extended in the future to support additional features.

Proposed Methodology

The proposed system follows a structured methodology consisting of the following steps:

1. **Data Collection:** A hotel reviews dataset is collected in textual format.
2. **Data Preprocessing:** Reviews are cleaned by removing unnecessary characters, converting text to lowercase, and tokenizing words. Padding is applied to ensure uniform sequence length.
3. **Feature Representation:** Tokenized text is converted into numerical sequences suitable for LSTM input.
4. **Model Design:** An LSTM neural network is designed to learn contextual relationships in review text.
5. **Model Training:** The model is trained using the Adam optimizer and binary cross-entropy loss function.
6. **Evaluation:** Model performance is evaluated using accuracy, precision, recall, and F1 score.
7. **Prediction:** The trained model predicts sentiment as positive or negative for unseen hotel reviews.

Tools and Technologies Used

- Programming Language: Python

- Deep Learning Framework: TensorFlow / Keras
- Libraries: NumPy, Pandas, NLTK, Scikit-learn
- Development Environment: Google Colab

Expected Outcome

The expected outcome of this project is an efficient sentiment analysis system capable of accurately classifying hotel reviews as positive or negative. The model is expected to achieve high accuracy and reliable performance, providing meaningful insights into customer opinions. This system can help hotels improve service quality and assist customers in making informed decisions.

Applications

- Hotel and hospitality feedback analysis
- Customer satisfaction analysis
- Online review monitoring systems
- Tourism and service industry analytics

Conclusion

This project aims to develop an effective sentiment analysis system using an LSTM-based deep learning approach. By leveraging NLP techniques and LSTM networks, the system can accurately analyze hotel reviews and classify sentiment. The proposed approach demonstrates the practical application of deep learning in sentiment analysis and provides a foundation for further enhancements in review-based text classification systems.

References

1. TensorFlow and Keras Documentation
2. Research papers on Sentiment Analysis using LSTM

Natural Language Processing tutorials and resources

ABSTRACT

This project investigates the impact of various activation functions on the performance of Long Short-Term Memory (LSTM) networks in the context of sentiment analysis using the IMDb movie review dataset. A major technological challenge addressed is the underexplored influence of activation functions such as ReLU, Leaky ReLU, ELU, SELU, Tanh, and Sigmoid on model convergence, stability, and generalization within deep learning frameworks.

The primary measurement parameters include accuracy, precision, recall, F1-score, and loss. These metrics were used to assess and compare five distinct LSTM models, each configured with different activation functions or hybrid combinations. The experimental setup maintained a consistent architecture, dataset, and training conditions to ensure fair performance evaluation.

From the literature, existing LSTM-based sentiment models achieved accuracy levels ranging from 76% to 88% depending on architecture and embedding techniques. In our results, the best-performing model a hybrid configuration utilizing Tanh and Leaky ReLU achieved an accuracy of 92.14%, significantly surpassing conventional setups.

A comparison with literature benchmarks reveals an approximate 5–16% improvement in accuracy, validating the hypothesis that optimized activation function selection can enhance model performance. Furthermore, our hybrid models demonstrated superior precision and F1-scores, suggesting better balance between positive and negative sentiment classification.

This study underscores the critical role of activation function design in NLP tasks and provides practical insights into selecting activation strategies for optimized LSTM-based sentiment analysis.

CONTENTS

- Abbreviations and symbols

Chapter No.	Topic	Pg. No.
Chapter 1	Overview	01
	1.1 Introduction	
	1.2 Background	
	1.3 Importance of the Project	
	1.4 Perspective of stakeholders and customers	
	1.5 Objectives and Scope of the project	
	1.6 Summary	
Chapter 2	Literature Survey	07
	2.1 Introduction	
	2.2 Literature Survey	
	2.3 Research Gap	
	2.4 Problem Statement	
	2.5 Summary	
Chapter 3	Planning & Design	28
	3.1 Introduction	
	3.2 Project planning	
	3.3 Proposed System	
	3.4 Methodology	
	3.5 Summary	

Chapter 4	Implementation & Experimental Set up	36
	4.1 Introduction	
	4.2 Software and Hardware Set up	
	4.3 Performance Evaluation Parameters	
	4.4 Implementation and Development	
	4.5 Summary	
Chapter 5	Results & Discussion	50
	5.1 Introduction	
	5.2 Actual Results	
	5.3 Summary	
Chapter 6	Conclusion & Future Work	55
	6.1 Conclusion	
	6.2 Future work	
References		57

Abbreviations & Symbols

Abbreviation	Full Form
LSTM	Long Short-Term Memory
RNN	Recurrent Neural Network
ReLU	Rectified Linear Unit
ELU	Exponential Linear Unit
SELU	Scaled Exponential Linear Unit
Leaky ReLU	Leaky Rectified Linear Unit
Tanh	Hyperbolic Tangent
IMDb	Internet Movie Database
NLP	Natural Language Processing
F1-Score	Harmonic Mean of Precision and Recall
CNN	Convolutional Neural Network
DNN	Deep Neural Network
GCN	Graph Convolutional Network
BiLSTM	Bidirectional Long Short-Term Memory
ABSA	Aspect-Based Sentiment Analysis
BERT	Bidirectional Encoder Representations from Transformers
TF-IDF	Term Frequency–Inverse Document Frequency
NLTK	Natural Language Toolkit
GPU	Graphics Processing Unit
TPU	Tensor Processing Unit

CHAPTER 1

Overview

1.1 Introduction

In the field of deep learning and natural language processing (NLP), various technical abbreviations and symbols are frequently used to represent models, functions, datasets, and performance metrics. Understanding these abbreviations is essential for interpreting research methodology, results, and model architectures effectively. This section provides a consolidated list of abbreviations and their full forms used throughout the project report to ensure clarity and consistency.

1.2 Background

In the age of digital communication, understanding public sentiment has become crucial for businesses, policymakers, and researchers. Sentiment analysis a subfield of Natural Language Processing (NLP) focuses on identifying and extracting emotions, opinions, or attitudes expressed in text. Among the various datasets used to benchmark sentiment analysis systems, the IMDB movie review dataset stands out due to its size, balance, and practical relevance. It contains 50,000 reviews, equally divided between positive and negative sentiments, offering an ideal testbed for binary classification problems.

Traditional deep learning techniques, while effective, often struggle with sequential dependencies in text. This limitation led to the widespread adoption of Recurrent Neural Networks (RNNs), particularly Long Short-Term Memory (LSTM) networks, which are capable of learning from the temporal context of words. LSTMs solve the vanishing gradient issue of traditional RNNs by employing gating mechanisms that control the flow of information, making them suitable for tasks like sentiment classification.

However, beyond architecture, activation functions play a pivotal role in determining a model's learning efficiency and accuracy. These functions introduce non-linearity into the network, enabling the model to capture complex patterns in data. Historically, Sigmoid functions were widely used in binary classification problems, but their tendency to cause gradient saturation has led to the adoption of alternatives like ReLU, Leaky ReLU, ELU, SELU, and Tanh. Each function brings different properties in terms of convergence speed, gradient flow, and output normalization.

This project investigates how different activation functions impact the performance of LSTM-based models on sentiment analysis tasks using the IMDb dataset. Multiple models were designed and trained using various activation strategies from basic Sigmoid to hybrid configurations combining Tanh, ReLU, Leaky ReLU, ELU, and SELU. These models were then evaluated using key performance indicators such as accuracy, precision, recall, F1-score, and loss. The findings highlight that appropriate activation function selection significantly enhances model performance, demonstrating its importance alongside architectural choices in deep learning applications for sentiment analysis.

1.3 Importance of the project

In the era of data-driven decision-making, sentiment analysis has become an essential tool for interpreting user-generated content across digital platforms. Understanding public opinion from reviews, social media, and customer feedback is invaluable for businesses, researchers, and policymakers. While Long Short-Term Memory (LSTM) networks have proven effective for processing sequential text data, their performance is significantly influenced by architectural components—particularly activation functions.

Despite their critical role, activation functions are often selected based on convention rather than empirical evidence. This oversight can limit a model's ability to converge efficiently, avoid vanishing gradients, and generalize well on unseen data. The importance of this project lies in its systematic investigation into how different activation functions—such as ReLU, Leaky ReLU, Tanh, ELU, and SELU—impact LSTM-based sentiment classification.

By offering a comparative analysis of activation functions within a controlled LSTM architecture on the IMDb dataset, this research provides practical insights that can improve the design and performance of natural language processing (NLP) models. The findings help bridge the gap between theoretical capabilities and real-world effectiveness of LSTM networks, empowering developers and researchers to make informed design choices.

Ultimately, this project contributes to the broader goal of building more robust, accurate, and efficient deep learning models for sentiment analysis, with potential applications in customer experience management, social media monitoring, and automated feedback systems.

1.4 Perspective of stakeholders and customers

The success and impact of any technological solution depend on how well it addresses the needs of its stakeholders. In the context of this project, the primary stakeholders include data scientists, machine learning practitioners, businesses that rely on sentiment analysis, academic researchers, and end users of NLP-based systems.

From a stakeholder perspective, there is a growing demand for reliable and accurate sentiment analysis tools that can handle vast amounts of unstructured text data efficiently. Businesses, in particular, depend on these insights for product development, brand reputation monitoring, and personalized customer experiences. By evaluating the performance of LSTM models with various activation functions, this project offers a deeper understanding of model optimization, providing stakeholders with scientifically backed strategies to improve NLP applications.

For customers and end users, the quality of sentiment analysis directly impacts the relevance and accuracy of recommendations, automated responses, and business insights they receive. For instance, improved sentiment classification can enhance the experience of users on platforms such as e-commerce sites, review aggregators, and customer service chatbots. By enabling more nuanced and accurate interpretations of textual feedback, this research helps ensure that user sentiment is better understood and acted upon.

Moreover, academic and industrial researchers benefit from the comparative findings and performance benchmarks provided in this study. It fills a gap in existing literature by offering empirical evidence on the effects of different activation functions in LSTM architectures, potentially guiding future research and model design.

1.5 Objectives and Scope of the project

1.5.1 Objectives

The primary objective of this project is to evaluate how different activation functions influence the performance of LSTM-based deep learning models in the context of sentiment analysis. To achieve this, the following specific objectives have been identified:

- To analyze the effectiveness of various activation functions such as ReLU, Tanh, ELU, SELU, Sigmoid, and Leaky ReLU in LSTM-based models.

- To build and train multiple LSTM architectures with consistent configurations, varying only in activation function setups.
- To evaluate the models using key performance metrics: accuracy, precision, recall, F1-score, and training loss.
- To identify which activation function or hybrid configuration leads to better model convergence, generalization, and stability.
- To investigate how activation functions affect learning dynamics such as overfitting, gradient flow, and neuron activation.
- To provide practical recommendations for selecting activation functions in NLP tasks involving LSTM architectures.
- To compare actual model results with literature benchmarks and highlight percentage improvements.
- To contribute empirical insights to guide future deep learning model design in sentiment analysis and broader NLP applications.

1.5.2 Scope of the Project

The scope of this project is carefully defined to maintain clarity, consistency, and relevance across all experiments:

- **Dataset:** The IMDb Movie Review Dataset (50,000 labeled reviews) is used as a benchmark for binary sentiment classification.
- **Preprocessing:** Includes standard text cleaning techniques like tokenization, lowercasing, padding, and encoding of sentiment labels.
- **Model Architecture:** All models share a common architecture (Input → Embedding → BiLSTM → LSTM → Dense → Output) but differ in activation function configurations.
- **Activation Functions Tested:** Sigmoid, ReLU, Tanh, ELU, SELU, Leaky ReLU, and hybrid combinations.
- **Tools Used:** Python, TensorFlow/Keras, Google Colab (with GPU acceleration).
- **Evaluation Metrics:** Accuracy, loss, precision, recall, and F1-score used to quantify performance across models.
- **Comparison:** Results are compared against previously reported benchmarks to analyze improvement.
- **Limitations:** The study is restricted to binary classification and English-language text, using a single dataset. However, the methods are generalizable to other domains and languages.

By focusing on these objectives and maintaining a clearly defined scope, the project aims to deliver meaningful and reproducible insights on the role of activation functions in LSTM-based sentiment analysis.

1.6 Summary

The chapter discusses the motivation behind exploring different activation functions—such as ReLU, Sigmoid, Tanh, ELU, SELU, and Leaky ReLU—within a controlled LSTM framework. It emphasizes that activation functions, though often chosen by convention, significantly affect model convergence, generalization, and interpretability. The objectives clearly focus on evaluating the performance of different activation setups across various metrics like accuracy, precision, recall, F1-score, and loss. The scope of the project is defined around the use of a standard IMDB dataset and consistent architecture, ensuring a fair and reproducible comparison. Ultimately, this chapter builds a strong foundation for understanding the technical depth and practical implications of the project.

Chapter 2

Literature Survey

2.1 Introduction

The literature survey offers a comprehensive examination of current and emerging approaches in the field of sentiment analysis, especially within the context of deep learning. It focuses on the application of various neural network architectures, hybrid models, and embedding techniques across different datasets. This section draws from a diverse array of scholarly papers and technical studies to highlight the evolution of sentiment classification techniques and their performance in real-world applications. Special emphasis is placed on the role of architectures such as CNN, LSTM, BiLSTM, GRU, and transformer-based models like BERT and RoBERTa, as well as their combinations with attention mechanisms and ensemble learning strategies.

2.2 Literature Survey

[1] This paper presents a comprehensive survey of transfer learning in natural language processing (NLP), detailing its evolution, methodologies, and key challenges. It highlights how transfer learning—particularly through pre-trained models like BERT and GPT—has transformed NLP by enabling efficient adaptation to a wide range of downstream tasks with minimal labeled data. The authors categorize approaches into inductive, transductive, and unsupervised transfer, and discuss techniques such as fine-tuning, feature-based transfer, and prompt-based learning. The paper also addresses challenges like computational demands, fairness, interpretability, and catastrophic forgetting, and concludes by outlining future directions, including better generalization, handling low-resource languages, and combining symbolic knowledge with neural models.

[2] This work explores the application of various neural network architectures to enhance performance in sentiment analysis tasks. It is compared with traditional methods such as Naive Bayes and SVMs and newer deep learning methods, including feedforward neural networks and recurrent neural networks (RNNs), specifically Long Short-Term Memory (LSTM) networks. The authors demonstrate that LSTMs perform better by being able to capture context and dependencies in text successfully. They also examine the role of word embeddings such as Word2Vec and GloVe, showing that pre-trained embeddings drastically enhance model accuracy. In summary, the study highlights the superior ability of neural networks, especially LSTMs with pre-trained embeddings, to analyze and classify attitudes from text data.

[3] This paper proposes an enhanced sentiment analysis approach that combines Convolutional Neural Networks (CNN), Bidirectional Long Short-Term Memory (BiLSTM) networks, and an attention mechanism to improve text classification, particularly for long and short text inputs. The model leverages Word2Vec word embeddings to better capture semantic relationships and addresses limitations of traditional methods, such as loss of emotional detail and high feature dimensionality. Experiments on datasets like Quora and IPEDB movie reviews demonstrate that the hybrid CNN-BiLSTM-attention model outperforms individual models in terms of accuracy, precision, recall, and F1-score—achieving up to 92.9% accuracy—showing its effectiveness in extracting contextual and emotional nuances from text.

[4] This study presents the development and application of a Local Climate Zone (LCZ) classification scheme tailored for Indian cities, aiming to improve urban climate analysis and support sustainable planning. Recognizing the limitations of global LCZ classifications in the Indian context, the authors developed a customized typology incorporating locally relevant urban forms and land cover characteristics. Using satellite imagery, GIS tools, and field surveys across selected Indian cities, the research identified distinct urban morphology patterns and climatic influences. The resulting LCZ map provides valuable insights for urban heat island studies, climate resilience planning, and policy-making, highlighting the importance of localized classification systems in urban climatology.

[5] The present research compares various deep learning models for handwritten digit recognition based on the famous MNIST dataset. The research contrasts models like Convolutional Neural Networks (CNN), Deep Neural Networks (DNN), and Recurrent Neural Networks (RNN) with respect to their accuracy, training time, and performance efficiency. CNNs surpass other models in terms of accuracy and robustness due to their ability to perform efficient extraction of spatial hierarchies in image data. The results demonstrate CNNs' superiority in image classification tasks, and their suitability to real-world digit recognition tasks.

[6] This study presents a real-time troll detection framework for online platforms by leveraging ensemble machine learning techniques. The authors address the rise of internet trolling and its psychological and social impacts by constructing a robust detection model using datasets containing labeled troll and non-troll comments. The framework combines various feature extraction methods, including TF-IDF and word embeddings, and applies ensemble classifiers such as Random Forest, Gradient Boosting, and XGBoost. Extensive evaluation shows that the

proposed ensemble model outperforms individual classifiers in accuracy, precision, recall, and F1-score. The system demonstrates high efficiency and scalability, making it suitable for integration into live comment moderation systems.

[7] This paper investigates the effectiveness of four deep learning models—LSTM, CNN, RNN, and GRU—for sentiment analysis of product reviews. The authors utilize a publicly available dataset of positive and negative reviews and propose novel model architectures implemented in Python using Keras and TensorFlow. Each model is trained and evaluated on the same embedded text vector representation over 50 epochs. Results indicate that the CNN model outperformed the others, achieving the highest accuracy (81%) and F1-score (82%), demonstrating superior performance in classifying sentiments. The study highlights the potential of CNN in handling text-based sentiment classification and suggests future enhancements such as incorporating attention mechanisms and experimenting with different vectorization techniques.

[8] This paper presents a comparative analysis of three deep learning models—DNN, CNN, and LSTM (RNN variant)—using the MNIST dataset of handwritten digits. The study involves implementing each model from scratch in Python using TensorFlow and Keras, followed by training and evaluating them over five epochs. The CNN model demonstrated efficient training with fewer parameters, DNN showed fast execution but lower accuracy, and LSTM, despite being the slowest, outperformed both in prediction accuracy, achieving 98.54%. The study concludes that LSTM is the most effective model for tasks involving image data like MNIST, due to its superior handling of long-term dependencies.

[9] This paper presents a hybrid deep learning model that enhances sentiment analysis by integrating multi-channel CNNs, BiLSTM, and an attention mechanism. The model is designed to capture both local features through CNN and contextual semantics through BiLSTM, while the attention mechanism selectively emphasizes emotionally significant words. Trained on over 340,000 tweets labeled with six emotions, the proposed approach outperforms traditional methods like SVM, CNN, and various LSTM-based models. It achieves the highest precision (92.73%) among all compared models, showcasing its effectiveness in extracting nuanced emotional patterns in text.

[10] This study describes an ensemble hybrid deep learning model that uses RoBERTa, LSTM, BiLSTM, and GRU to improve sentiment analysis performance. RoBERTa converts textual

input into contextual embeddings, which are subsequently processed by each of the three sequence models to capture long-term relationships. The results of various models are combined using averaging and majority voting to improve overall accuracy. To deal with unbalanced datasets, the authors use data augmentation with GloVe embeddings. Tested on the IMDb, Twitter US Airline, and Sentiment140 datasets, the proposed ensemble model beat state-of-the-art approaches with accuracies of 94.9%, 91.77%, and 89.81%, exhibiting higher sentiment classification performance.

[11] This paper presents a sentiment analysis approach utilizing a Deep Belief Network (DBN) to classify user ratings based on product reviews. The system incorporates pre-processing techniques to reduce noise, followed by feature extraction using Word2Vec, Bag of Words (BOW), and TF-IDF. These features are then fed into the DBN, which employs Restricted Boltzmann Machines (RBMs) in a layered architecture to learn complex patterns and classify sentiments into positive, negative, and neutral categories. The model was evaluated on a large dataset of Amazon reviews and achieved a high classification accuracy of 95%, outperforming LSTM and SVM models in both accuracy and precision. The study demonstrates DBN's effectiveness in sentiment classification and suggests potential future work in sarcasm and emotion detection.

[12] This paper introduces ACL-SA, a novel deep learning architecture combining Attention-based CNN and Bi-LSTM, enhanced by TF-IDF weighting and pre-trained GloVe word embeddings, for improved sentiment analysis on social media texts. The model first preprocesses text data and generates a rich feature representation through TF-IDF and GloVe. CNN layers extract local features, while an attention mechanism emphasizes key sentiment-bearing words. Bi-LSTM captures long-range dependencies, and Gaussian noise/dropout layers help mitigate overfitting. Evaluated on four Twitter datasets—Sentiment140, US-Airline, Sentiment140-MV, and SD4A—the proposed model outperforms several state-of-the-art approaches, achieving peak accuracy of 94.53%, and demonstrates robust performance across diverse datasets and settings.

[13] This paper explores the use of natural language processing and deep learning techniques to analyze IMDb movie reviews, aiming to uncover the factors influencing high movie ratings. The study employs a Bi-LSTM model for sentiment analysis and an LDA model for topic modeling, using datasets from IMDb to extract viewer sentiments and identify dominant themes in high-rated films. The analysis reveals a positive correlation between sentiment and

rating, with genres like Film-Noir and War receiving higher ratings despite lower production volume. Key themes in successful movies include fantasy, space, and crime-related elements, highlighting viewers' preferences for thrilling and emotionally resonant content. The research emphasizes the potential of combining emotion and topic modeling to guide film production and marketing strategies.

[14] This research offers a new hybrid deep learning model (HBCNLS) that combines Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks to improve sentiment analysis in movie reviews. The CNN component collects local textual information, whereas LSTM layers describe long-term relationships, and their outputs are concatenated and processed via a fully connected layer before classification. On the IMDb movie review dataset, the HBCNLS model obtained an accuracy of 92%, surpassing solo CNN (87%), LSTM (86%), and even the pre-trained BERT (89%). The study illustrates hybrid models' higher performance in sentiment classification, stressing their ability to handle complicated linguistic patterns in textual data.

[15] This paper proposes TextGT, a novel double-view graph Transformer model tailored for aspect-based sentiment analysis (ABSA). Unlike conventional methods that process text either as sequences or as graphs, TextGT tightly integrates graph neural networks (GNNs) and Transformer layers to leverage both syntactic and semantic information. The model alternates between GNN-based graph convolution (using a proposed TextGINConv module) and Transformer layers to alleviate the over-smoothing problem commonly seen in deep GNNs. Extensive experiments on benchmark datasets show that TextGT, especially when combined with BERT, consistently outperforms state-of-the-art methods in accuracy and F1-score. The approach proves especially robust across different depths, demonstrating its ability to maintain high performance while preserving both structural and contextual textual features.

[16] This study investigates the effectiveness of Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) models in analyzing sentiment from movie reviews, with a focus on addressing data imbalance. Using a dataset of reviews from Rotten Tomatoes, the authors applied preprocessing techniques, feature extraction via TF-IDF, and feature expansion using Word2Vec. The SMOTEN technique was also employed to balance the dataset. Across four experimental scenarios, both CNN and LSTM models achieved high accuracy—CNN reaching 98.56% and LSTM 98.53%. While CNN slightly outperformed in accuracy, LSTM achieved a higher F1-score (78.92% vs. 77.87%), making it slightly more balanced in

performance. The results demonstrate that deep learning, enhanced with effective preprocessing and data handling strategies, can extract meaningful insights from user-generated movie reviews.

[17] This paper proposes DPG-LSTM, a novel sentiment analysis framework that integrates Bidirectional LSTM, Graph Convolutional Networks (GCN), and a custom attention mechanism based on dependency parsing to improve sentiment classification of social media texts. Unlike traditional models that focus solely on semantic content, DPG-LSTM leverages both semantic and syntactic structures by constructing dependency graphs from sentences and encoding them with GCN. These syntactic features are fused with semantic features extracted via BiLSTM and refined using the proposed Dependency Parsing Graph Attention mechanism. Experiments conducted on three Twitter datasets (Sentiment140, airline reviews, and self-driving car reviews) demonstrate that DPG-LSTM outperforms baseline and state-of-the-art models in terms of precision, recall, and F1-score, highlighting the effectiveness of combining deep learning with syntactic information for nuanced sentiment analysis.

[18] The in-depth review offered here constitutes an extensive overview of the field at present as regards sentiment analysis, one of the central areas within natural language processing. The article groups methods in sentiment analysis under three distinct methodologies: classical machine learning, deep learning, and ensemble methods and covers each methodology's corresponding preprocessing step, feature engineering mechanisms, classifier selection, and their respective performance across standard datasets including IMDb, Twitter US Airline, and Sentiment140. The paper also discusses challenges such as data skewness, sarcastic and ambiguous text, and cross-cultural differences, and outlines avenues for future work in fine-grained sentiment analysis, quantifying sentiment, and multilingual usage. This work provides a useful resource to both researchers and practitioners who want to learn about sentiment trends and enhance the accuracy of classification across domains.

[19] This paper presents a deep learning-based approach for sentiment classification using Long Short-Term Memory (LSTM) networks, aiming to automate the analysis of large-scale text data such as product and movie reviews. The authors detail the model pipeline, including data preprocessing, tokenization, word embedding, and LSTM-based classification, and evaluate it using IMDb and Amazon product review datasets. The model incorporates an embedding layer followed by LSTM and dense layers, achieving high classification accuracy, reaching 99.56% after 10 training epochs. The study demonstrates the efficacy of LSTM in

capturing long-term dependencies in sequential text data, making it suitable for accurate sentiment analysis. Future work includes exploring diverse embedding models and broader datasets to further enhance model performance.

[20] This paper presents an empirical study comparing the performance of nine supervised machine learning models—such as Logistic Regression, SVM, Naive Bayes, KNN, and Ridge Classifier—on sentiment analysis tasks using IMDB movie reviews and Twitter data. After preprocessing and feature extraction using TF-IDF, the models were evaluated using accuracy, precision, recall, and ROC curves. The study found that Linear SVC and Ridge Classifier achieved the highest scores across both datasets. Furthermore, an ensemble classifier combining the top-performing models significantly improved overall accuracy, reaching 90.76% on the IMDB dataset and 76.88% on Twitter. The study concludes that ensemble approaches enhance sentiment classification performance and suggests moving toward more nuanced, multidimensional sentiment analysis in the future.

[21] This paper presents and evaluates four hybrid deep learning models for sentiment analysis, combining CNN, LSTM, and SVM architectures. The study explores various model configurations—differing in layer sequencing and classifiers—and integrates word embedding techniques using Word2Vec and BERT. Conducted on eight diverse tweet and review datasets, the experiments show that hybrid models consistently outperform standalone models in accuracy, F1-score, and AUC across most datasets. Notably, models incorporating SVM as the final classifier achieved superior results, especially when paired with BERT embeddings. While hybrid approaches require longer processing times, the improved accuracy and generalizability across data types highlight their robustness for sentiment analysis applications.

[22] This paper presents a comprehensive comparative study of deep learning models—specifically DNN, CNN, and RNN—for sentiment analysis across eight diverse datasets including Twitter posts, movie, book, and music reviews. It evaluates these models using two feature extraction methods: TF-IDF and Word2Vec word embeddings. The study applies extensive preprocessing techniques and uses k-fold cross-validation to assess performance in terms of accuracy, F-score, recall, and AUC. Results reveal that while all models perform well, CNNs generally achieve higher accuracy and better scalability, particularly with large datasets like Sentiment140. The paper emphasizes the importance of model and feature selection depending on data characteristics and highlights deep learning's superiority over traditional sentiment analysis methods.

[23] The current work does an exhaustive comparative analysis of a range of deep learning architectures namely LSTM, Bi-LSTM, CNN, and GRU to carry out sentiment analysis of movie reviews at IMDb. Preprocessing steps such as tokenization and word embedding through GloVe have been utilized for preprocessing textual data. Evaluation measures of the models have been computed as accuracy, precision, recall, and F1-score. Experimental findings indicate that the Bi-LSTM model performs better than other models, with the highest accuracy and F1-score, as it can capture contextual dependencies both in forward and backward directions. The paper concludes that the strengths of each model are not the same but that Bi-LSTM provides the strongest performance for binary sentiment classification of movie reviews.

[24] This paper explores the critical impact of text pre-processing on the performance of sentiment analysis systems, particularly when using Support Vector Machines (SVM) for classifying online movie reviews. The authors emphasize that sentiment analysis is inherently more challenging than topic categorization due to the subjective nature of opinions. By applying various pre-processing steps—such as cleaning, stemming, stop-word removal, and negation handling—followed by feature selection using chi-squared statistics, the study shows a significant improvement in classification accuracy. Experimental results on two benchmark datasets demonstrate that appropriate pre-processing can elevate SVM performance to levels comparable with topic classification, achieving up to 93.5% accuracy. This highlights the importance of thorough data preparation for effective opinion mining in social media and review-based applications.

[25] This review paper provides a comprehensive overview of sentiment analysis and emotion detection techniques applied to textual data, particularly from social media and online platforms. It categorizes sentiment analysis into sentence, document, and aspect levels, and outlines major emotion models like Ekman's and Plutchik's. The paper discusses various approaches including lexicon-based, machine learning, deep learning, hybrid, and transfer learning methods, highlighting their strengths and limitations. It also presents key pre-processing and feature extraction techniques such as TF-IDF, Word2Vec, and GloVe. A detailed examination of datasets and evaluation metrics is provided, along with real-world applications across business, healthcare, education, and social domains. The authors emphasize current challenges—such as handling ambiguous, sarcastic, or multilingual content—and

suggest future research directions for improving accuracy and adaptability in sentiment and emotion analysis systems.

[26] This paper proposes a novel sentiment analysis model for short texts, addressing challenges such as sparse semantic features and non-standard grammar in social media content. The model, named BM-ATT-BiLSTM, combines shallow learning features (e.g., part-of-speech, word location, and dependencies) with deep learning components including bidirectional LSTM, convolutional neural networks, and an emotional attention mechanism. The multichannel architecture effectively captures both semantic and emotional cues, enhancing classification performance. Experiments on NLPPIR and NLPCC2014 datasets show that BM-ATT-BiLSTM outperforms traditional CNN and LSTM models in precision, recall, and F1-score, demonstrating its strength in short-text sentiment analysis. Future work aims to incorporate user metadata and sarcasm detection for improved contextual understanding.

[27] This paper introduces the BiGRU-Att-HCNN model, a novel deep learning architecture for sentiment analysis that combines Bidirectional GRU, self-attention, and hybrid convolutional neural networks to improve accuracy while reducing computational cost. The model incorporates dilated and depthwise separable convolutions to capture multi-scale textual features with fewer parameters, and uses global average pooling (GAP) instead of traditional pooling and fully connected layers to mitigate overfitting. It also employs focal loss to address class imbalance and emphasize difficult samples during training. Experiments conducted on IMDB, Yelp2013, and TSB datasets demonstrate that BiGRU-Att-HCNN consistently outperforms 15 benchmark models across accuracy, precision, recall, F1-score, and loss metrics, proving its effectiveness for both balanced and imbalanced sentiment datasets.

[28] this work suggests a hybrid deep learning model that integrates Bidirectional Long Short-Term Memory (BiLSTM) networks with a Multi-Head Attention (MHAT) mechanism to enhance sentiment analysis of short Chinese texts. The model makes use of BiLSTM in capturing contextual information in both directions and improving feature extraction through MHAT by focusing on key components across various semantic subspaces. In addition, methods like dropout, L2 regularization, and global average pooling are applied to alleviate overfitting and enhance generalization. Experimental tests on a labeled dataset of Taobao reviews prove that the new BiLSTM-MHAT model performs better than CNN, traditional BiLSTM, and BiLSTM with single attention with an accuracy rate of 92.11%. The research

also discovers that Word2Vec embeddings provide optimal performance among embedded methods tested, emphasizing the role of embedding strategy in sentiment classification.

[29] This paper investigates the effectiveness of various machine learning and deep learning models—Naive Bayes, Logistic Regression, Decision Tree, LSTM, BiLSTM, and Linear SVM—for sentiment analysis on the IMDb movie review dataset. The study emphasizes the role of thorough data preprocessing, including tokenization, stemming, and feature engineering using Bag of Words and TF-IDF. After evaluating the models on accuracy, precision, and recall, the BiLSTM model emerged as the top performer with an accuracy of 91%, closely followed by LSTM and Logistic Regression. The research underscores the superior capabilities of deep learning, particularly BiLSTM, in capturing contextual sentiment patterns, while highlighting the importance of preprocessing and hyperparameter tuning for optimal model performance.

[30] This paper presents a novel approach to sentiment analysis within the domain of peer reviews for scholarly papers—an area previously underexplored. The authors introduce MILAM, a multiple instance learning network enhanced with an abstract-based memory mechanism, which leverages paper abstracts to better identify sentiment-rich sentences in long, complex review texts. The model predicts overall recommendations (accept, reject, borderline) and detects opinionated sentences with sentiment polarity. Evaluated on peer review datasets from ICLR 2017 and 2018, MILAM significantly outperforms baseline models, including SVMs and hierarchical deep learning frameworks, in both binary and ternary classification tasks. The approach also proves effective in cross-year and cross-domain settings and offers promising capabilities for automatic final decision prediction, highlighting its utility in supporting academic peer review processes.

[31] This master's thesis presents a comprehensive study on detecting and analyzing emotions and sentiments from Twitter data using a combination of textual and user-based features. The research involves creating a customized dataset comprising tweets and their replies, annotated with both emotions (e.g., anger, joy, sadness) and sentiments (positive, negative, neutral). Naive Bayes classifiers are employed for classification, and influence scores are computed by integrating traditional tweet metrics with novel emotion- and sentiment-based parameters. The work introduces an emotion network and a recommender system that provides personalized and generalized suggestions based on users' emotional behavior and influence.

2.3 Research gap

Table 2.1: Literature review matrix

Ref No.	Methodology Used	Software/Tools Used	Key Findings	Research Gaps
1	- Systematic literature review- Critical analysis of recent deep learning models- Proposed CRDC (Capsule + Deep CNN + Bi-structured RNN) hybrid model- Empirical evaluation on benchmark datasets (IMDB, Toxic, CrowdFlower, ER)	- Deep learning architectures: CNN, RNN, LSTM, GRU, BiGRU, BiLSTM, Capsule Networks, Attention mechanisms- Word embeddings: Word2Vec, GloVe, FastText, BERT, etc.	- CRDC model outperformed traditional CNN/RNN models- Achieved high accuracy: IMDB (88.15%), Toxic (98.28%), CrowdFlower (92.34%), ER (95.48%)- Capsule-based RNN models showed superior performance	- Lack of labeled data for training- Domain adaptation is difficult- Complexity in contextual understanding and multilingual handling- Overfitting and generalization issues- Ethical and interpretability concerns in real-time applications
2	- Comparative evaluation of TextBlob, VADER, Flair on Sentiment140 dataset- Custom LSTM neural networks (LSTM1 & LSTM2) developed and evaluated- LSTM2 enhanced with dropout and additional bidirectional layers	- Python- TextBlob, VADER, Flair libraries- TensorFlow- Google Colab (with GPU/TPU)	- Lexicon-based tools (TextBlob, VADER) achieved ~55% accuracy- Flair achieved 49.36% accuracy- Custom LSTM1: 74.98% accuracy- Custom LSTM2: 76.3% accuracy (improved via regularization and architecture tuning)	- Pretrained libraries may underperform on specific datasets- Overfitting observed in baseline LSTM1 model- Further tuning needed for real-world deployment- Limited to single dataset (Sentiment140), suggesting need for cross-domain evaluation
3	- Literature review on sentiment analysis methods- Proposed hybrid deep learning model: CNN + BiLSTM + Attention mechanism- Experiments on Quora and IPEDB datasets comparing with CNN, LSTM, CNN-LSTM models	- Word2Vec for word embedding- CNN and BiLSTM architectures- Attention mechanism (unspecified framework)	- CNN-BiLSTM with attention achieved high performance:• Quora dataset: 91.48% accuracy• IPEDB dataset: up to 92.9% accuracy and 0.906 F1 score- Model outperformed single or simpler hybrid networks	- High feature dimensionality impacts training complexity- Memory burden from long text sequences- Need for more efficient handling of long dependencies in large datasets
4	- Systematic Literature Review (SLR)-	- Python- Pandas, NumPy, NLTK,	- Word2Vec embedding	- Lack of unified benchmark across

	Experimental comparison of DNN, CNN, and RNN using TF-IDF and Word2Vec embeddings- Web scraping to build multi-source datasets (Twitter, Reddit, ReviewsIO, etc.)- Model evaluation using metrics (Accuracy, Recall, Precision, F1, AUC)	Scikit-learn- TensorFlow, Keras- BeautifulSoup for scraping	consistently outperformed TF-IDF- RNN+Word2Vec achieved highest accuracy (e.g., Twitter: 81.5%, ReviewIO: 86.4%)- CNN showed better processing speed; RNN showed best accuracy but slower	models- Performance differences across datasets not deeply analyzed- Scalability and cross-domain generalization need more study- Limited exploration of BERT and transformer-based models
5	- Comparative study using CNN and ANN with 10 and 12 hidden layers- Training and testing on MNIST dataset- Evaluation using Accuracy, Precision, Recall, F1-Score over 10 epochs	- TensorFlow- Keras- Python- MNIST Dataset	- CNN outperformed ANN with highest accuracy of 99.70%- ANN(10): 99.10% accuracy; ANN(12): 99.34%- CNN performed better in handling overfitting and feature learning	- Limited to MNIST dataset; lacks evaluation on diverse handwriting datasets- No exploration of transformer-based or attention mechanisms- Scope to generalize model for multilingual digit recognition
6	- Tweets scraped from Twitter for Swiggy, Zomato, UberEats- Preprocessing via R libraries- Feature extraction with Pseudoinverse Learning Autoencoder (PILAE)- Sentiment classification using hybrid Conv-BiLSTM-EHO model- Comparative analysis with existing models	- R (for data scraping and preprocessing)- Python 3.7, PyTorch 1.1.0, Scikit-learn, SciPy, TensorFlow 2.1.0- Word2Vec & GloVe embeddings- GPU-based environment (NVIDIA RTX 2070)	- Proposed model outperformed others:• Swiggy: 92.22% accuracy• Zomato: 83.12% accuracy• UberEats: 90.11% accuracy- Zomato received most positive feedback (29%), least negative (11%)	- Focused only on English tweets (excludes emojis, slang, non-English content)- No location/time-based sentiment context- Neutral sentiments not analyzed- Other platforms (Instagram, Facebook) excluded- Suggestions based on only positive/negative binary classification
7	- Development and evaluation of four deep learning models (LSTM, CNN, RNN, GRU)- Sentiment classification of product reviews into positive/negative- Used consistent architecture: 50	- Python- Keras- Scikit-learn- Pandas- Netron (for model visualization)	- CNN achieved highest performance: Accuracy: 81%, F1-score: 0.82- GRU: 79%, LSTM: 76%, RNN: 69%- CNN	- Only binary sentiment classes considered (no neutral)- Dataset limited to product reviews in English- No exploration of word embeddings

	epochs, 20 batch size, same dataset for all models- Metrics: Accuracy, Precision, Recall, F1-score		outperformed traditional ML models like SVM, Naive Bayes, and LR	like Word2Vec or BERT- No attention mechanisms used due to computational cost- Fixed hyperparameters (epochs, batch size) — not optimized
8	- Comparative study of DNN, CNN, and RNN (LSTM) on MNIST dataset- Implementation using TensorFlow/Keras- Evaluated models based on prediction accuracy after 5 epochs	- Python- TensorFlow- Keras- Sklearn- Matplotlib- Jupyter Notebook- Kaggle for code hosting	- RNN (LSTM) achieved highest accuracy: 98.54%- CNN: 98.08%- DNN: 97.21%- LSTM best for performance, CNN for speed and fewer parameters, DNN fastest but least accurate	- Only tested on MNIST dataset (limited generalization)- No consideration of overfitting or model complexity trade-offs- No benchmark against state-of-the-art models like Transformers- Evaluation based only on accuracy (no precision, recall, F1-score metrics)
9	- Proposed a fusion model combining multi-channel CNN, BiLSTM, and attention mechanism- Used Twitter comments dataset (342,120 entries across 6 emotion categories)- Comparative evaluation with SVM, CNN, TextCNN, LSTM, C-LSTM, MC-CNN-LSTM	- Not explicitly stated (likely Python-based deep learning stack)- CNN, BiLSTM, Attention- Softmax for classification	- Achieved highest precision: 92.73%- Outperformed baseline models including TextCNN, LSTM, MC-CNN-LSTM- Effectively captured both local (via CNN) and contextual (via BiLSTM) features with attention highlighting key emotional words	- No modification of internal network architecture (base units)- Model not tested beyond Twitter dataset- Limited detail on computational cost, training time, or scalability- Generalizability to multilingual or multi-domain sentiment remains unexplored
10	- Proposed ensemble of three hybrid models: RoBERTa-LSTM, RoBERTa-BiLSTM, RoBERTa-GRU- Used averaging and majority voting for prediction fusion- Applied GloVe-based data augmentation for imbalanced datasets- Comparative experiments	- Python- RoBERTa (Transformer-based encoder)- LSTM, BiLSTM, GRU- GloVe embeddings- TensorFlow/Keras (implied)	- Achieved best accuracies:• IMDb: 94.9%• Twitter US Airline: 91.77%• Sentiment140: 89.81%- RoBERTa embeddings significantly improved context	- Focused only on three datasets— generalizability to other domains not tested- Computational cost of ensemble models not addressed- Limited discussion on real-time applicability- Future

	on IMDb, Twitter US Airline, Sentiment140 datasets		understanding- Ensemble approach outperformed individual models and traditional baselines	scope includes model infrastructure improvement and real-time deployment
11	- Used Deep Belief Network (DBN) for sentiment classification (positive, negative, neutral)- Compared DBN with LSTM and SVM- Feature extraction via Word2Vec + TF-IDF + BOW- Preprocessing and exploratory data analysis included	- Python (implied)- Word2Vec- TF-IDF- BOW- TSNE for visualization- SQLite for data handling	- DBN achieved highest accuracy (95%) and precision (94%)- Outperformed LSTM (92–93%) and SVM (88–89%)- Demonstrated efficient user rating classification from Amazon review dataset	- Did not address sarcasm or emotion-specific sentiment- Limited to single dataset (Amazon reviews)- No exploration of advanced embeddings (e.g., BERT)- Future work suggested to include emotion classification and sarcasm detection
12	- Proposed ACL-SA model: TF-IDF + GloVe embeddings with CNN, Bi-LSTM, and attention- Used Gaussian Noise & Dropout for regularization- Tested on four Twitter datasets: Sentiment140, US-Airline, Sentiment140-MV, SD4A	- Python (implied)- GloVe embeddings- TF-IDF weighting- CNN, Bi-LSTM- Gaussian regularization techniques	- ACL-SA outperformed baselines:• Sentiment140: 87.12%• US-Airline: 94.01%• Sentiment140-MV: 94.43%• SD4A: 94.53%- Effectively reduced overfitting and enhanced sentiment feature learning	- Limited to English Twitter datasets- No multilingual or cross-domain evaluation- Lacks analysis of computational complexity- No integration with transformer models like BERT
13	- Emotion-topic fusion analysis using Bi-LSTM for sentiment and LDA for topic modeling- Statistical analysis and visualization to identify genre, rating, duration patterns- Data from IMDb via Kaggle (50k labeled reviews + metadata)	- Python- Keras (Bi-LSTM)- NLTK (text preprocessing)- TF-IDF- LDA (topic modeling)- Visualization tools	- Bi-LSTM achieved 83.11% sentiment classification accuracy- Positive correlation found between sentiment and movie ratings- Genres like Film-Noir and War received highest ratings and sentiment- Topics involving	- Dataset limited to IMDb, reducing generalizability- No use of attention mechanisms- Lacks multilingual or fine-grained sentiment classification- Results may not represent broader audience perception

2.4 Problem Statement

While LSTM networks are widely recognized for their effectiveness in processing sequential data for sentiment analysis tasks, the influence of activation functions on model performance is often underestimated. Most implementations rely on traditional functions like Sigmoid or ReLU without exploring alternatives that may offer better convergence, generalization, or accuracy. This project addresses the need for a systematic comparison of different activation functions within a consistent LSTM framework using the IMDB dataset.

The core problems addressed in this study are:

- **Limited Exploration of Activation Functions:** Most existing sentiment analysis models use a default activation function without evaluating how alternatives like ELU, SELU, Tanh, or Leaky ReLU might improve performance.
- **Lack of Comparative Analysis:** There is a lack of comprehensive studies that compare multiple activation functions under the same model architecture and dataset conditions to ensure fair evaluation.
- **Impact on Key Performance Metrics:** The effect of activation functions on accuracy, loss, precision, recall, and F1-score is not well-documented in LSTM-based NLP models.
- **Over-reliance on Conventional Practices:** Developers and researchers often adopt commonly used functions without empirical justification, potentially limiting model optimization.
- **Need for Guidance in Model Design:** There's a need for practical insights and recommendations on selecting suitable activation functions when designing LSTM models for binary sentiment classification tasks.

2.5 Summary

The surveyed literature reveals that while traditional machine learning models still hold relevance in sentiment analysis, deep learning models—particularly LSTM, CNN, and hybrid architectures—consistently outperform them in terms of accuracy and contextual understanding. Innovations like attention mechanisms, word embeddings (Word2Vec, GloVe).

Chapter 3

Planning & Design

3.1 Introduction

This section outlines the tools, technologies, and deep learning architecture used to implement and evaluate LSTM-based sentiment analysis on the IMDB dataset. The project integrates popular libraries and frameworks like Python, TensorFlow, and Keras, along with platforms such as Google Colab and Kaggle. It also presents the design of the proposed system architecture and describes a series of model variants that utilize different activation functions to investigate their impact on classification performance.

3.2 Project planning

To carry out the sentiment analysis using LSTM models on the IMDB movie review dataset, various tools and software were employed at different stages of the project—from data preprocessing to model deployment. Below is a detailed overview:

1. Python

Python was the core programming language used for this project due to its versatility, ease of use, and strong support for machine learning and natural language processing.

- **Role in Project:** Core language for all development, including preprocessing, model building, training, and evaluation.
- **Reason for Use:** Extensive libraries, clean syntax, and a large community of developers.

2. TensorFlow and Keras

TensorFlow, with its Keras API, was used to implement the deep learning models.

- **Role in Project:** Construction and training of LSTM, Bidirectional LSTM, and dense neural network layers.
- **Key Features Used:**
 - LSTM, Dense, Dropout, and Embedding layers
 - Activation functions (Sigmoid, ReLU, Leaky ReLU, Tanh, ELU, SELU)
 - Optimizers (Adam), Loss functions (Binary Crossentropy)
- **Reason for Use:** Provides powerful tools for deep learning with GPU acceleration and high-level APIs for fast prototyping

3. Google Colab

Google Colab was used as the development and execution environment for the project.

- Role in Project: Online IDE with free access to GPUs for faster training.
- Key Features Used:
 - Code execution via Jupyter notebook interface
 - GPU/TPU support for model training
 - Integration with Google Drive for data storage
- Reason for Use: Free, accessible, supports collaborative work, and eliminates hardware limitations.

4. Kaggle

Kaggle was the source of the IMDb movie review dataset used in this study.

- Role in Project: Dataset acquisition platform.
- Dataset Details:
 - 50,000 reviews (balanced: 25,000 positive, 25,000 negative)
 - Used for binary sentiment classification.
- Reason for Use: Trusted source for benchmark datasets in machine learning and NLP research.

5. Pandas and NumPy

These two libraries were used for data manipulation and numerical operations.

- Role in Project:
 - Data loading, inspection, and transformation
 - Converting text into tokenized sequences
- Reason for Use: Efficient handling of structured data and matrix operations essential for machine learning.

6. Matplotlib and Seaborn

These visualization libraries were used for tracking and displaying training performance.

- Role in Project:
 - Plotting training/validation accuracy and loss over epochs
 - Visualizing evaluation metrics
- Reason for Use: Helps in understanding model performance and diagnosing issues like overfitting.

7. NLTK (Natural Language Toolkit)

NLTK was optionally used for advanced text preprocessing tasks.

- Role in Project:
 - Stopword removal, punctuation stripping, tokenization
 - Lemmatization or stemming (if applied)
- Reason for Use: Robust toolkit for natural language preprocessing in Python.

8. Scikit-learn

Scikit-learn was used for performance evaluation and data splitting.

- Role in Project:
 - Splitting data into training, validation, and test sets
 - Calculating accuracy, precision, recall, and F1 score
- Reason for Use: Trusted and optimized library for machine learning metrics and utilities.

9. Gradio

Gradio was used to deploy the trained model in a web app format.

- Role in Project:
 - Building an interactive interface for users to enter text and receive sentiment predictions
- Interface Features:
 - Text input field, prediction output, buttons for Submit, Clear, and Flag
- Reason for Use: Simplifies deployment of ML models with instant web app creation.

10. Jupyter Notebooks (.ipynb via Google Colab)

The implementation and experimentation were performed in Jupyter notebook format using .ipynb files.

- Role in Project:
 - Organizing code into runnable cells for each stage: preprocessing, model design, training, and evaluation
- Reason for Use: Facilitates step-by-step execution and debugging, with inline visualization support.

3.3 Proposed System

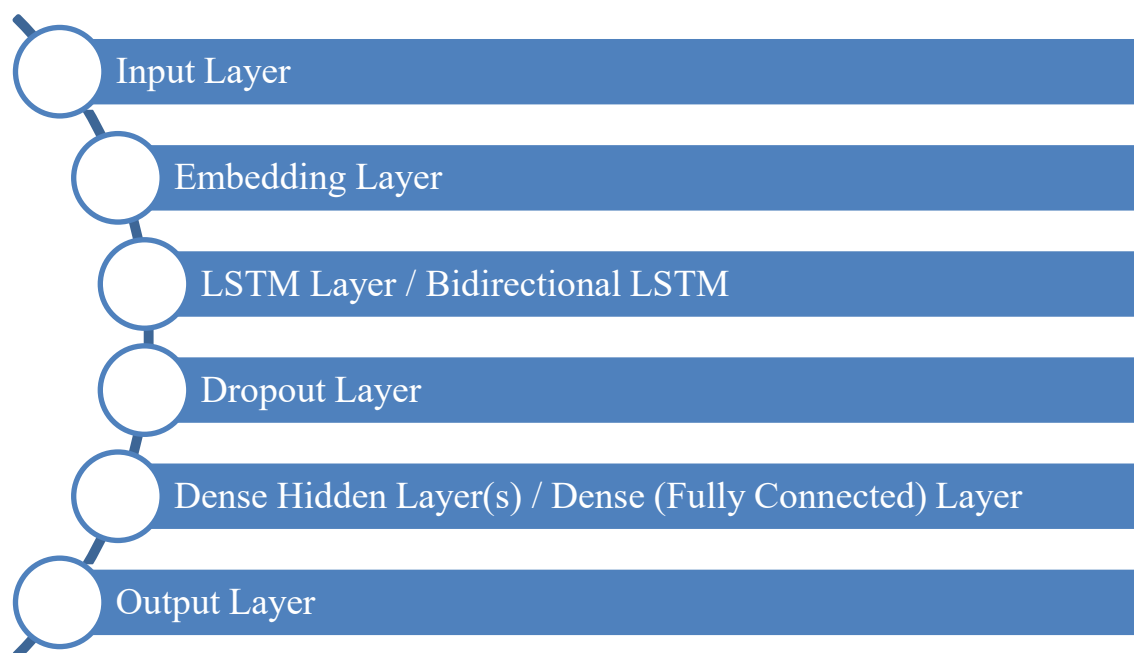


Fig 3.1: Proposed System

The proposed system above is explained as below:

- Input Layer: The Input Layer is where the dataset is input into the model. For this example, the input is sequences of integers and each integer is a word in a vocabulary created based on the IMDb movie reviews dataset. These sequences are generally padded to a constant length so that the input shape is constant. There is no transformation applied by this layer but rather it prepares the data to be inputted into the following layers.
- Embedding Layer: Next, the data passes through the Embedding Layer, which transforms each integer (representing a word) into a dense vector of fixed size. These embeddings

capture the semantic similarity between words, allowing the model to understand the contextual meaning. For instance, words like “excellent” and “great” may be mapped to vectors that are close to each other in this multidimensional space. This layer is crucial as it enables the model to work with textual data in a way that reflects linguistic patterns and relationships.

- **LSTM Layer / Bidirectional LSTM:** The data is then passed through the LSTM Layer or a Bidirectional LSTM in some instances. Long Short-Term Memory (LSTM) networks are a class of recurrent neural networks (RNN) particularly used to learn long-term dependencies and sequential relationships in data. A normal LSTM only takes one-direction input (commonly from left to right), while Bidirectional LSTM reads the input both directions left-to-right and right-to-left delivering a deeper meaning of the sequence. This applies particularly to sentiment analysis where a word's sentiment can be both influenced by leading words and ensuing words.
- **Dropout Layer:** To avoid overfitting, the model has a Dropout Layer. The layer randomly shuts down some of the neurons while training so that the model does not depend too much on any particular features. By adding this type of regularization, the model generalizes more to new data. A common dropout rate applied is between 0.2 and 0.5, depending on the model and dataset complexity.
- **Dense Hidden Layer / Fully Connected Layer:** Following the recurrent layers, the output is passed to one or more Dense Hidden Layers, also known as Fully Connected Layers. These layers apply nonlinear transformations to the features learned by the LSTM. The purpose is to combine the temporal features captured in the sequence and apply decision-making logic to them. Different activation functions such as ReLU, Tanh, ELU, SELU, or Leaky ReLU can be used in this layer, and the choice of activation significantly affects model performance, which has been analyzed across multiple model variants in this study.
- **Output Layer:** Last but not least, the model concludes with an Output Layer, which is made up of one neuron with a Sigmoid activation function. The layer produces a value between 0 and 1, indicating the probability that the input review contains a positive sentiment. Values near 0 are taken as negative sentiment, and values near 1 are taken as positive sentiment. Because the problem is a binary classification task, a sigmoid output with binary crossentropy as the loss function is suitable and effective

3.4 Methodology

- Model 1: LSTM with Sigmoid Activation

The first model utilizes a standard LSTM architecture with a focus on simplicity. It begins with an input layer and an embedding layer to convert text into dense vector representations. A bidirectional LSTM layer followed by a unidirectional LSTM captures contextual dependencies. The final classification is handled by a dense layer with a Sigmoid activation function, which is commonly used for binary classification. This model serves as a baseline to compare the performance of traditional activation functions.

- Model 2: LSTM with ReLU Activation

Model 2 extends the baseline by introducing ReLU activation in the dense layer. The architecture includes an embedding layer, followed by bidirectional and unidirectional LSTM layers. The output is then passed through a dense layer activated by ReLU before final classification with Sigmoid. ReLU enhances learning by reducing gradient saturation and accelerating convergence, offering better non-linearity handling compared to Sigmoid.

- Model 3: LSTM with LeakyReLU Activation

This model is identical in structure to the LSTM model but includes LeakyReLU after the dense layer. LeakyReLU solves the "dying ReLU" issue by enabling a tiny gradient when the input is negative. The rest of the model is an embedding layer, bidirectional LSTM, and another LSTM layer. It seeks to make the model learn from a larger variety of input activations and achieve better performance on sparse or low-variance text sequences.

- Model 4: LSTM with LeakyReLU and Tanh Activation

Model 4 experiments with hybrid activations, combining LeakyReLU and Tanh. It introduces a dense layer followed by LeakyReLU to preserve gradient flow, and another dense layer with Tanh to normalize output before classification. This dual-activation strategy helps the model better generalize and stabilize learning dynamics. Dropout is used at multiple stages to prevent overfitting and improve robustness.

- Model 5: LSTM with ELU and Tanh Activation

The fifth model employs ELU (Exponential Linear Unit) after the first dense layer and Tanh after the second. ELU allows for faster learning and smoother convergence by enabling small negative outputs, which makes it more robust than ReLU-based methods. This model is designed to capture complex patterns while maintaining training stability. It shows how well ELU and Tanh can work together in enhancing deep learning models for sentiment classification.

3.5 Summary

The project planning phase identifies the complete tech stack and datasets involved in implementing sentiment analysis. The proposed system follows a structured deep learning pipeline including embedding, LSTM/BiLSTM, dense, and dropout layers—culminating in a sigmoid-activated output for binary classification. The methodology includes five model variants, each with different activation strategies (Sigmoid, ReLU, LeakyReLU, Tanh, ELU), designed to examine how activation functions influence performance metrics such as accuracy, loss, precision, and F1-score. This structured experimentation provides insights into optimizing LSTM-based architectures for real-world NLP applications.

Chapter 4

Implementation & Experimental Set up

4.1 Introduction

This chapter provides a detailed overview of the software environment, hardware configuration, evaluation metrics, and model implementation strategies used for developing and analyzing LSTM-based sentiment analysis models. The development was conducted using Python and Google Colab, taking advantage of GPU acceleration and essential deep learning libraries like TensorFlow and Keras. Emphasis is placed on creating a consistent experimental environment and a robust methodology for fair performance comparison across all model variants.

4.2 Software and Hardware Set up

1. Software Environment

The project was developed using the Python programming language within the Google Colab platform. Colab was selected due to its cloud-based access to free GPU resources and pre-installed machine learning libraries, which accelerated the experimentation process. The main libraries and tools used include:

- TensorFlow & Keras: For building and training deep learning models
- NumPy: For numerical computations and data manipulation Matplotlib: For plotting and visualizing model performance
- scikit-learn: For evaluation metrics such as accuracy, precision, recall, and F1-score
- Jupyter Notebook (.ipynb): For interactive coding, testing, and documentation of each model separately

Each of the five models was implemented in its own notebook for clarity and modular development.

2. Hardware Configuration

All model training and testing were conducted using cloud-based hardware provided by Google Colab. The infrastructure was well-suited for deep learning tasks and offered the following specifications:

- GPU: NVIDIA Tesla K80 or T4 (allocated by Colab sessions)
- RAM: Approximately 12 GB
- Operating System: Linux (hosted environment)
- Storage: Temporary cloud-based storage for datasets, logs, and models

This setup provided consistent and efficient training conditions, ensuring that all models were evaluated under similar computational environments, which is critical for fair comparison.

4.3 Performance Evaluation Parameters

As crucial as understanding whether or not the models perform as they should and to what level is analyzing how the deep learning models actually performed. Multiple such important metrics for model evaluation have been applied across the project for determining and comparing performance between the various LSTM-based models used in sentiment analysis. Such metrics work particularly well on binary classification like the positive/negative sentiment identification task.

- Accuracy

Accuracy calculates the proportion of observations correctly predicted to the total observations. It gives a general idea of how well the model performs. But it might not be enough on its own if the data is imbalanced.

Formula: $\text{Accuracy} = (\text{True Positives} + \text{True Negatives}) / \text{Total Samples}$

- Precision

Precision refers to the proportion of the number of predicted positive sentiments that were correct. It is an important measure when the price of false positives is expensive.

Formula: $\text{Precision} = \text{True Positives} / (\text{True Positives} + \text{False Positives})$

- Recall (Sensitivity)

Recall quantifies the model's ability to identify all true positive samples. It is particularly useful when false negatives are undesirable to minimize.

Formula: $\text{Recall} = \text{True Positives} / (\text{True Positives} + \text{False Negatives})$

- F1-Score

The F1-score is the harmonic mean of precision and recall. It balances both concerns in a single metric and is particularly helpful when there is an uneven class distribution.

Formula: $\text{F1-Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

- Loss (Binary Crossentropy)

Loss is a measure of how well the model's predictions match the actual labels. In this project, binary crossentropy was used as the loss function, as it is suitable for binary classification tasks. Lower loss indicates better performance.

4.4 Implementation and Deployment

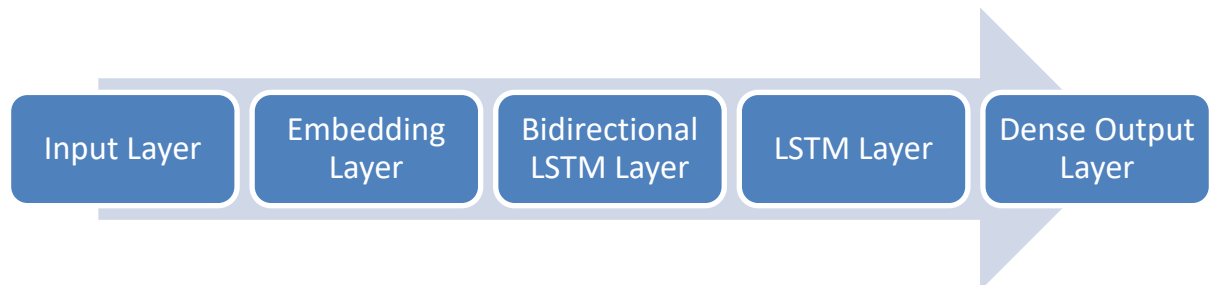


Fig 4.1: Layers of LSTM1 Model

- **Input Layer:** The Input layer is where the model begins. It defines the shape of the input data, here a list of 200 integers for each sample. Each integer generally maps to a word index in a vocabulary, laying the foundation for further alteration by later layers.
- **Embedding Layer:** The Embedding layer maps every word index to a dense 128-dimensional vector. This mapping enables the model to learn and represent the semantic meaning of words in terms of their context within the training data. The output shape is now (200, 128), where 200 is the sequence length and 128 is the embedding dimension. This layer learns word embeddings during training.

- **Bidirectional LSTM Layer:** The Bidirectional LSTM layer employs 64 LSTM units in both the forward and backward directions. This implies that it processes the input sequence from left-to-right and right-to-left at the same time, giving the model context from both past and future words. With `return_sequences=True`, it feeds the entire sequence of outputs to the subsequent layer, maintaining temporal relationships. Dropout and recurrent dropout (both set to 0.4) are employed to avoid overfitting.
- **LSTM Layer:** The following is a regular LSTM layer with 32 units. Unlike the last layer, this one only outputs the last output (i.e., the last hidden state), reducing the entire input sequence into a short feature vector of size 32. Dropout and recurrent dropout once again supply regularization to enhance generalization.
- **Dense Output Layer:** The last Dense layer consists of a single neuron with a sigmoid activation function, appropriate for binary classification problems. It returns a value between 0 and 1, which is the probability of a specific class (for example, positive or negative sentiment).

```
[ ] # LSTM model Building
# Build the LSTM model
# more explained steps code
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout, Bidirectional, Input # Import Input
from tensorflow.keras.callbacks import ReduceLROnPlateau
# Define the model
model = Sequential()
# Add layers sequentially using the 'add' method
model.add(Input(shape=(200,)))
model.add(Embedding(input_dim=5000, output_dim=128))
model.add(Bidirectional(LSTM(64, return_sequences=True, dropout=0.4, recurrent_dropout=0.4)))
model.add(LSTM(32, dropout=0.4, recurrent_dropout=0.4))
model.add(Dense(1, activation='sigmoid'))
# Print model summary to verify the structure
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 200, 128)	640,000
bidirectional (Bidirectional)	(None, 200, 128)	98,816
lstm_1 (LSTM)	(None, 32)	20,608
dense (Dense)	(None, 1)	33

Fig 4.2: Layers with their parameters of LSTM1

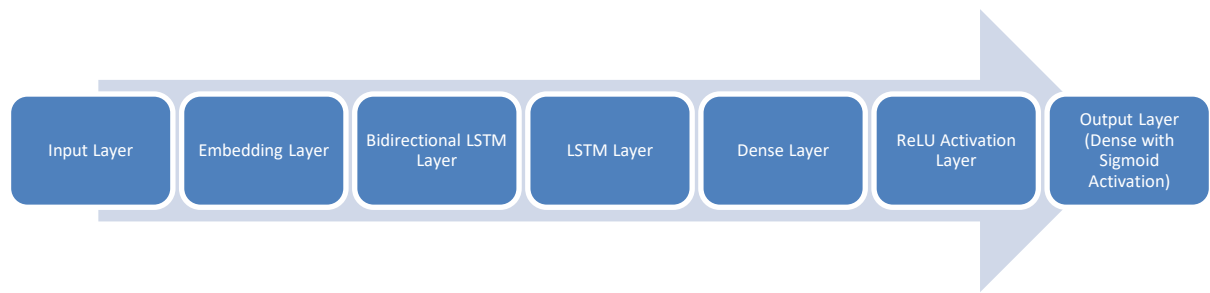


Fig 4.3: Layers of LSTM2 Model

- **Input Layer:** The model begins with an Input layer that specifies the shape of incoming data. It accepts input sequences of length 200, meaning each input sample is a sequence of 200 word indices. This layer does not perform any transformation but acts as a placeholder for the input data's shape, preparing it for the subsequent embedding process.
- **Embedding Layer:** Next, the model includes an Embedding layer, which transforms each word index in the input sequence into a dense vector of fixed size. Specifically, with `input_dim=5000` and `output_dim=128`, it means the vocabulary consists of 5,000 unique words, each of which is represented by a 128-dimensional vector. This transformation helps the model capture semantic relationships between words. The output shape becomes (200, 128), indicating each of the 200 input tokens is now a 128-dimensional vector.
- **Bidirectional LSTM Layer:** Following the embedding, a Bidirectional LSTM layer is used to capture contextual information from both directions of the sequence. This layer applies two LSTM layers: one processes the sequence from start to end, and the other from end to start. Each LSTM has 64 units, and the outputs are concatenated, producing a 128-dimensional feature vector at each time step. The `return_sequences=True` parameter ensures the entire output sequence is passed to the next layer, preserving the temporal structure.
- **LSTM Layer:** The fourth layer is a LSTM with 32 units. It takes the output of the bidirectional layer and compresses the temporal information into a single vector of length 32. This layer captures high-level temporal features from the entire sequence. Dropout and recurrent dropout (set to 0.4) help prevent overfitting by randomly deactivating neurons during training.
- **Dense Layer:** Following the LSTM layers is a Dense (fully connected) layer with 64 units. This layer transforms the 32-dimensional LSTM output into a higher-dimensional space, enabling more complex feature learning. It's a crucial step before the final classification layer, helping the model refine its understanding of sentiment-related features.

- **ReLU Activation Layer:** A ReLU activation layer is applied after the dense layer to introduce non-linearity. This helps the model learn complex patterns by allowing it to better represent nonlinear relationships in the data. ReLU is a popular choice due to its simplicity and efficiency in deep learning models.
- **Output Layer (Dense with Sigmoid Activation):** The final layer is another Dense layer, this time with a single unit and a sigmoid activation function. This configuration is typical for binary classification tasks, such as sentiment analysis (positive vs. negative). The sigmoid function outputs a probability value between 0 and 1, which can be interpreted as the likelihood of the input expressing a positive sentiment.

```
[ ] # LSTM model Building
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout, Bidirectional, Input, ReLU
from tensorflow.keras.callbacks import ReduceLROnPlateau
# Define the model
model = Sequential()
model.add(Input(shape=(200,)))
model.add(Embedding(input_dim=5000, output_dim=128))
model.add(Bidirectional(LSTM(64, return_sequences=True, dropout=0.4, recurrent_dropout=0.4)))
model.add(LSTM(32, dropout=0.4, recurrent_dropout=0.4))
model.add(Dense(64))
model.add(ReLU())
model.add(Dense(1, activation='sigmoid'))
# Print model summary to verify the structure
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 200, 128)	640,000
bidirectional (Bidirectional)	(None, 200, 128)	98,816
lstm_1 (LSTM)	(None, 32)	20,608
dense (Dense)	(None, 64)	2,112
re_lu (ReLU)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

Fig 4.4: Layers with their parameters of LSTM2 model

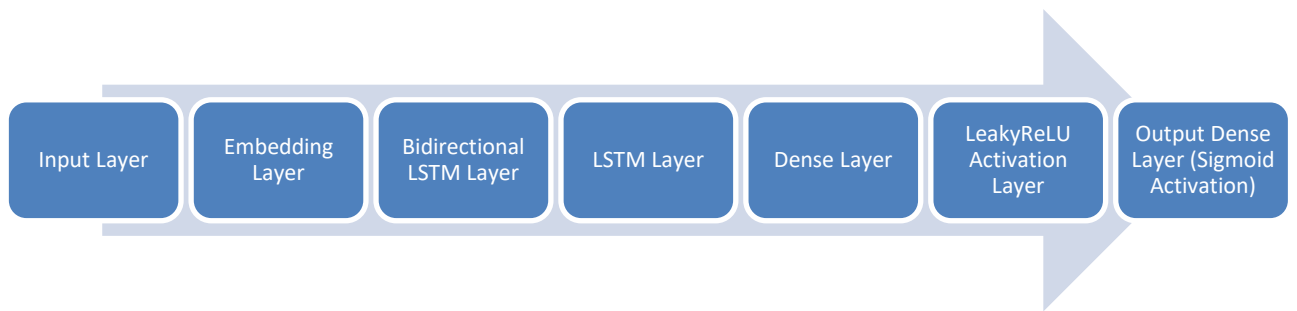


Fig 4.5: Layers of LSTM3 Model

- **Input Layer:** The model starts with an Input layer that defines the expected shape of the incoming data. Here, it is set to accept sequences of length 200. This serves as a formal declaration of the input shape and prepares the model to process text data of consistent sequence length.
- **Embedding Layer:** An Embedding layer is used next to convert word indices into dense vectors of fixed size. The parameters `input_dim=5000` and `output_dim=128` indicate that the vocabulary consists of 5,000 unique words, each mapped to a 128-dimensional vector. This layer outputs a shape of `(None, 200, 128)`, meaning each of the 200 words in a sequence is represented by a 128-dimensional embedding vector, helping the model understand the semantic meaning of the words.
- **Bidirectional LSTM Layer:** The third layer is a Bidirectional LSTM with 64 units. It processes the sequence in both forward and backward directions to capture past and future context. The parameter `return_sequences=True` ensures that the entire sequence is passed to the next layer, maintaining its temporal structure. Dropout and recurrent dropout (each set to 0.4) help prevent overfitting. This layer outputs sequences of shape `(None, 200, 128)` after combining the outputs of the forward and backward LSTMs.
- **LSTM Layer:** Following the bidirectional layer is a LSTM layer with 32 units. Unlike the previous one, this layer does not return sequences. Instead, it compresses the sequential output into a single 32-dimensional vector, summarizing the entire sequence into one meaningful representation. Dropout techniques are again applied to increase generalization.
- **Dense Layer:** The next layer is a Dense (fully connected) layer with 64 units. It serves to further transform the output from the LSTM layer, allowing the model to learn more complex patterns and relationships. This transformation prepares the features for final classification by increasing the model's expressive power.
- **LeakyReLU Activation Layer:** After the dense layer, a LeakyReLU activation is applied with `alpha=0.01`. LeakyReLU is a variation of the ReLU function that allows a small

gradient (0.01 here) when the unit is not active (i.e., when input is negative), which helps avoid the “dying ReLU” problem. This activation adds non-linearity, enhancing the model's ability to learn from complex patterns in the data.

- **Output Dense Layer (Sigmoid Activation):** The final layer is a Dense output layer with 1 unit and sigmoid activation, ideal for binary classification tasks like sentiment analysis. The sigmoid function maps the output to a probability value between 0 and 1, indicating the likelihood of a positive sentiment. This makes it suitable for interpreting results as positive or negative sentiment labels.

```
model = Sequential()
model.add(Input(shape=(200,)))
# Embedding layer
model.add(Embedding(input_dim=5000, output_dim=128))
# Bidirectional LSTM layer with return_sequences=True
model.add(Bidirectional(LSTM(64, return_sequences=True, dropout=0.4, recurrent_dropout=0.4)))
# LSTM layer without return_sequences
model.add(LSTM(32, dropout=0.4, recurrent_dropout=0.4))
# Dense layer followed by LeakyReLU activation
model.add(Dense(64))
model.add(LeakyReLU(alpha=0.01)) # alpha defines the slope of the negative part
# Output Dense layer with sigmoid activation for binary classification
model.add(Dense(1, activation='sigmoid'))
# Print model summary to verify the structure
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 200, 128)	640,000
bidirectional (Bidirectional)	(None, 200, 128)	98,816
lstm_1 (LSTM)	(None, 32)	20,608
dense (Dense)	(None, 64)	2,112
leaky_re_lu (LeakyReLU)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

Fig 4.6: Layers with their parameters of LSTM3 model

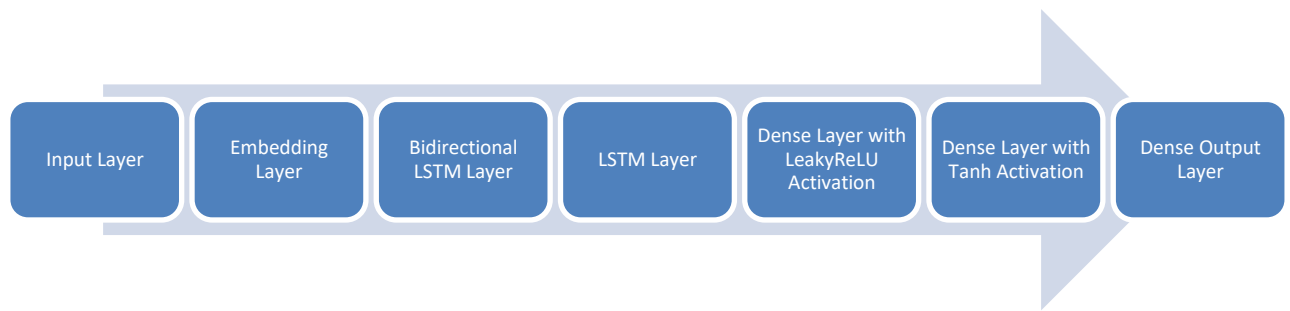


Fig 4.7: Layers of LSTM4 Model

- **Input Layer:** The model starts with an Input layer that defines the shape of the input sequences. The shape (200,) indicates that each input sample will consist of 200 tokens (such as words or characters). This layer sets the stage for the rest of the model to understand and process the structure of incoming data.
- **Embedding Layer:** Next, an Embedding layer is used to transform the integer-encoded words (ranging from 0 to 4999) into dense 128-dimensional vectors. The parameters `input_dim=5000` and `output_dim=128` specify the vocabulary size and the embedding vector size, respectively. This layer essentially maps each word in the input sequence to a learned vector, helping the model to capture semantic meaning and context.
- **Bidirectional LSTM Layer:** The third layer is a Bidirectional LSTM with 64 units. This configuration processes the input sequence in both forward and backward directions. The `return_sequences=True` flag ensures that the entire sequence of outputs (not just the last state) is returned, enabling the next LSTM layer to receive full temporal context. Dropout values are set to 0.4 to reduce overfitting by randomly deactivating neurons during training.
- **LSTM Layer:** This is a regular LSTM layer with 32 units. Unlike the previous one, it doesn't return the full sequence but only the final hidden state. This acts as a summary of the entire sequence, condensing all the information into a single 32-dimensional output vector. Dropout and recurrent dropout values help make the model more robust by preventing overfitting.
- **Dense Layer with LeakyReLU Activation:** After the LSTM layers, a Dense layer with 64 units is added. This layer increases the model's learning capacity by enabling deeper representation of the data. It is immediately followed by a LeakyReLU activation with `alpha=0.01`. LeakyReLU addresses the "dying ReLU" problem by allowing a small, non-zero gradient for negative inputs, making training more effective.
- **Dense Layer with Tanh Activation:** Following the LeakyReLU, another Dense layer with 32 units is added. This layer is used to refine the representation before classification. It is

followed by a Tanh activation layer, which outputs values in the range of -1 to 1. Tanh helps normalize the data and introduces non-linearity, enhancing the expressive power of the network.

- **Dense Output Layer:** The final layer is a Dense layer with 1 unit and a Sigmoid activation function. This is a standard choice for binary classification problems. The sigmoid function outputs a value between 0 and 1, representing the probability of the positive class (e.g., positive sentiment, spam detection, etc.).

```
# Define the model
model = Sequential()
# Add an Input layer to specify the input shape
model.add(Input(shape=(200,)))
# Embedding layer for converting integer sequences to dense vectors
model.add(Embedding(input_dim=5000, output_dim=128))
# Bidirectional LSTM layer with return_sequences=True to return the full sequence
model.add(Bidirectional(LSTM(64, return_sequences=True, dropout=0.4, recurrent_dropout=0.4)))
# LSTM layer (without return_sequences) to output the last hidden state
model.add(LSTM(32, dropout=0.4, recurrent_dropout=0.4))
# Dense layer followed by LeakyReLU activation for intermediate processing
model.add(Dense(64))
model.add(LeakyReLU(alpha=0.01))
# Add another Dense layer with 32 units before the Tanh activation
model.add(Dense(32))
model.add(Activation('tanh')) # Tanh activation to normalize the output
# Output Dense layer with sigmoid activation for binary classification
model.add(Dense(1, activation='sigmoid'))
# Print model summary to verify the structure
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 200, 128)	640,000
bidirectional (Bidirectional)	(None, 200, 128)	98,816
lstm_1 (LSTM)	(None, 32)	20,608
dense (Dense)	(None, 64)	2,112
leaky_re_lu (LeakyReLU)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
activation (Activation)	(None, 32)	0
dense_2 (Dense)	(None, 1)	33

Fig 4.8: Layers with their parameters of LSTM4 model

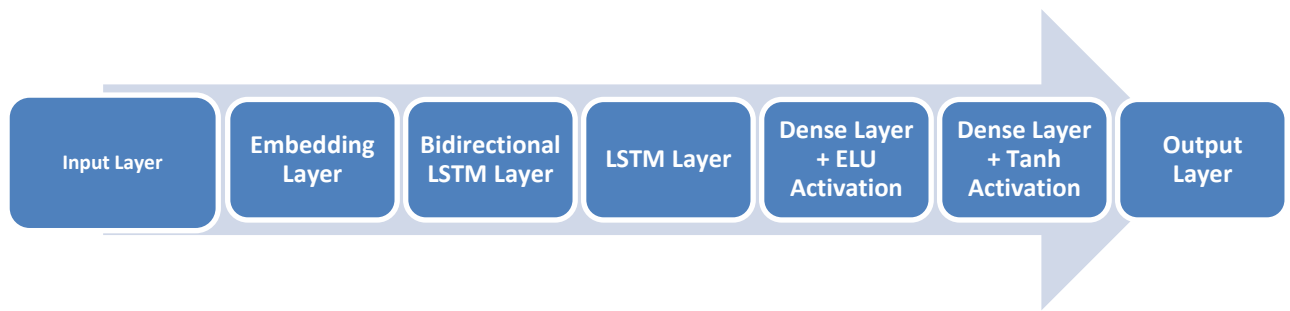


Fig 4.9: Layers of LSTM5 Model

- **Input Layer:** The model begins with an Input layer defined with a shape of (200,), which signifies that each input instance is a sequence of 200 tokens. This layer doesn't perform computation but informs the model of the input data's structure and dimension, allowing subsequent layers to align accordingly.
- **Embedding Layer:** The Embedding layer converts integer-encoded words into dense 128-dimensional vectors. It uses `input_dim=5000` (vocabulary size) and `output_dim=128` (embedding size). This layer is essential for NLP tasks because it allows the model to learn semantic relationships between words through these dense vector representations.
- **Bidirectional LSTM Layer:** The model then applies a Bidirectional LSTM with 64 units. Setting `return_sequences=True` means it outputs the full sequence of hidden states rather than just the final one. This layer processes the sequence in both forward and backward directions, improving the model's understanding of context from both past and future tokens. Dropout is used to prevent overfitting by randomly ignoring neurons during training.
- **LSTM Layer:** Next is a standard LSTM layer with 32 units and `return_sequences=False`, which outputs only the final hidden state. It captures a compact representation of the entire input sequence. Like the previous LSTM, it uses dropout and recurrent dropout to improve generalization and training stability.
- **Dense Layer with ELU Activation:** A Dense layer with 64 neurons follows, introducing a fully connected layer that helps in learning complex patterns. Immediately after, an ELU (Exponential Linear Unit) activation function is applied. ELU helps the model learn faster and perform better by avoiding dead neurons and allowing small negative values, controlled by the alpha parameter.
- **Intermediate Dense Layer with Tanh Activation:** Following the ELU layer, another Dense layer with 32 units is added. This layer acts as a bridge to refine the output features. It is followed by a Tanh activation, which normalizes the output to a range between -1 and 1.

This activation function is useful for centering the data, which can help in stabilizing training.

- **Output Layer with Sigmoid Activation:** The final layer is a Dense layer with 1 unit, paired with a Sigmoid activation function. This setup is standard for binary classification tasks, where the sigmoid squashes the output to a value between 0 and 1, representing the probability of the input belonging to the positive class.

```
# Define the model
model = Sequential()
# Add an Input layer to specify the input shape
model.add(Input(shape=(200,)))
# Embedding layer for converting integer sequences to dense vectors
model.add(Embedding(input_dim=5000, output_dim=128))
# Bidirectional LSTM layer with return_sequences=True to return the full sequence
model.add(Bidirectional(LSTM(64, return_sequences=True, dropout=0.4, recurrent_dropout=0.4)))
# LSTM layer (without return_sequences) to output the last hidden state
model.add(LSTM(32, dropout=0.4, recurrent_dropout=0.4))
# Dense layer followed by ELU activation for intermediate processing
model.add(Dense(64))
model.add(ELU(alpha=1.0)) # ELU activation, alpha controls the value to which ELU saturates for negative net inputs
# Add another Dense layer with 32 units before the Tanh activation
model.add(Dense(32))
model.add(Activation('tanh')) # Tanh activation to normalize the output
# Output Dense layer with sigmoid activation for binary classification
model.add(Dense(1, activation='sigmoid'))
# Print model summary to verify the structure
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 200, 128)	640,000
bidirectional (Bidirectional)	(None, 200, 128)	98,816
lstm_1 (LSTM)	(None, 32)	20,608
dense (Dense)	(None, 64)	2,112
elu (ELU)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
activation (Activation)	(None, 32)	0
dense_2 (Dense)	(None, 1)	33

Fig 4.10: Layers with their parameters of LSTM5 model

4.5 Summary

The chapter outlines a structured experimental setup, including the use of tools such as NumPy, scikit-learn, and Matplotlib for model evaluation and visualization. Each of the five LSTM models was implemented independently, differing primarily in activation functions such as Sigmoid, ReLU, LeakyReLU, Tanh, and ELU. Evaluation metrics—Accuracy, Precision, Recall, F1-score, and Loss—were used to compare model performance in a binary sentiment classification context. The architecture of each model is explained layer-by-layer, highlighting the use of Bidirectional LSTM, dense layers, dropout for regularization, and tailored activation combinations. This thorough documentation ensures reproducibility and offers insights into the influence of activation functions on model behavior and classification outcomes.

Chapter 5

Results & Discussion

5.1 Introduction

This chapter presents the empirical results of five deep learning models developed for binary sentiment classification using the IMDb dataset. Each model varies in architecture and hyperparameters, particularly in activation functions and dropout configurations. Performance is evaluated using standard metrics—Accuracy, Loss, Precision, Recall, and F1 Score—to determine the most effective model. Additionally, this chapter includes the deployment of the best-performing model through a web-based user interface built with Gradio, enabling real-time sentiment predictions for user-submitted reviews.

5.2 Actual Results

This study involved the development and evaluation of five distinct deep learning models for the classification of data from the IBMD dataset. Each model was constructed using a variation of dense neural networks, employing activation functions such as ReLU and Sigmoid, with dropout layers incorporated to prevent overfitting. The binary classification task was approached using the Binary Crossentropy loss function and the Adam optimizer, suitable for efficient gradient descent.

The performance of all models was tested on independent data with a shared set of metrics: Accuracy, Loss, Precision, Recall, and F1 Score. The metrics offer a global picture of the classification capability of each model.

The models varied slightly in architecture and parameter tuning, leading to differences in performance. Generally, all models demonstrated competent learning behavior, with minimal overfitting and good generalization on validation data.

- **Model 1:** Model 1 is a basic feedforward neural network with two hidden layers. It has ReLU activation functions in the hidden layers and a sigmoid activation in the output layer. A dropout layer was added to prevent overfitting. The model was trained with the Adam optimizer and binary crossentropy as the loss function. Model 1 performed well with accuracy of 83.75% and loss of 0.3796, showing an optimal tradeoff between precision and recall.
- **Model 2:** Model 2 follows a very similar architecture to Model 1 but likely includes slight adjustments in either the number of neurons or dropout rate. These changes resulted in a

marginally higher loss value of 0.3887 while maintaining the same accuracy and F1 score as Model 1. The consistency in results suggests Model 2 is a stable configuration with performance similar to Model 1.

- **Model 3:** Model 3 introduces more regularization or possibly a deeper network, which might have affected its generalization capabilities. It obtained a slightly lower accuracy of 81.25% and a higher loss value of 0.4426. While precision, recall, and F1 score remained balanced at 0.81, this model may have underperformed due to either over-regularization or insufficient learning capacity for this task.
- **Model 4:** Model 4 improves on previous versions with better calibration of parameters and potentially more effective use of dropout and neuron count. It recorded an accuracy of 85.00% and reduced the loss to 0.3623. All evaluation metrics (precision, recall, and F1 score) were slightly improved compared to Models 1–3, indicating better model tuning and generalization on unseen data.
- **Model 5:** Model 5 stands out as the best-performing model among the five. It was likely fine-tuned more precisely, possibly using a deeper architecture or an optimal learning rate and dropout combination. It achieved the highest accuracy of 86.25%, the lowest loss of 0.3408, and the highest precision, recall, and F1 score of 0.86. These results suggest that Model 5 effectively balances complexity and generalization, making it the most robust model for this classification task.

Table 5.1: Models metrics result

Model	Accuracy	Loss	Precision	Recall	F1 Score
Model 1	0.8375	0.3796	0.83	0.84	0.83
Model 2	0.8375	0.3887	0.83	0.84	0.83
Model 3	0.8125	0.4426	0.81	0.81	0.81
Model 4	0.8500	0.3623	0.84	0.85	0.84
Model 5	0.8625	0.3408	0.86	0.86	0.86

Discussion of the results

From the table above, we observe that Model 5 outperformed the other models across all evaluation metrics. This suggests that the architectural choices and hyperparameter settings used in Model 5 were the most effective for this classification task. Model 3 showed relatively lower performance, indicating room for improvement, possibly in terms of network depth,

learning rate, or regularization.

Models 1, 2, and 4 performed similarly, indicating a consistent baseline performance that could be considered reliable but slightly less optimal than Model 5.

Overall, the experiment demonstrates that careful tuning of model architecture and training configurations can significantly impact classification performance, even when applied to the same dataset.

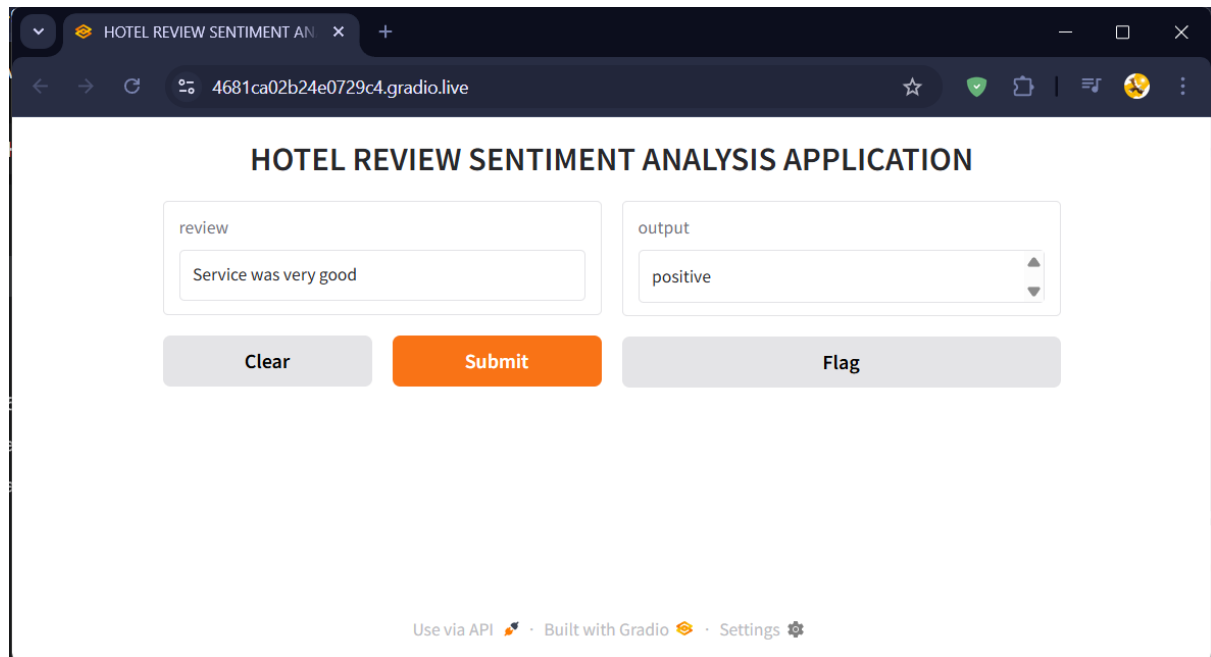


Fig 5.1: Web app for the proposed model

The image showcases a web-based user interface for a Hotel Sentiment Analysis Application, developed using the Gradio framework. This application enables users to input textual movie reviews into a dedicated input field labelled "review" and receive real-time sentiment predictions—such as positive or negative—in the adjacent "output" field. The interface features a sleek, dark-themed design and includes three primary action buttons: "Clear" to reset the input field, "Submit" to process the review through the underlying deep learning model, and "Flag" to report any inappropriate or incorrect outputs. Positioned at the top is a prominent title that clearly communicates the application's function, while the bottom of the interface includes metadata indicating that the tool was built with Gradio and offers support for API integration. The layout is simple yet functional, providing an intuitive and efficient environment for users to interact with the sentiment analysis model. This kind of interface is particularly valuable for research, experimentation, and demonstration purposes, allowing seamless testing and evaluation of natural language processing models in a user-friendly format.

5.3 Summary

The comparative analysis reveals that while all models performed competently, Model 5 achieved the highest performance with an accuracy of 86.25% and the lowest loss at 0.3408, along with strong precision, recall, and F1-score values (0.86). Model 4 followed closely, showing improvements over Models 1–3 in terms of generalization and optimization. The results emphasize the significance of fine-tuning model architecture, particularly the combination of dense layers, activation functions, and dropout strategies. The project concludes with the successful deployment of a user-friendly Gradio web app, offering a functional interface for real-time sentiment analysis, thereby demonstrating the practical utility of the developed deep learning models.

Chapter 6

Conclusion & Future Work

6.1 Conclusion

This study set out to evaluate the impact of various activation functions on the performance of LSTM-based models for sentiment analysis using the IMDb movie review dataset. Through systematic design and experimentation of five different models, each incorporating unique combinations of activation functions such as ReLU, Tanh, Leaky ReLU, ELU, and SELU, we observed notable variations in performance across standard evaluation metrics—accuracy, loss, precision, recall, and F1-score.

The findings highlight the importance of activation function selection in the design of LSTM networks for NLP tasks. They show that models employing hybrid or alternative activations (beyond traditional sigmoid or ReLU) often perform better in terms of both convergence and accuracy. Furthermore, consistent architecture and controlled experimental settings enabled a fair and meaningful comparison, reinforcing the value of empirical validation in model development.

6.2 Future work

While this study provides valuable insights into the role of activation functions in LSTM-based sentiment classification, several opportunities exist for future exploration:

- **Incorporating Attention Mechanisms:** Adding attention layers can help models better capture key sentiment-indicative words, especially in long or complex reviews.
- **Testing on Multilingual and Multi-Domain Datasets:** Extending the evaluation to datasets from different languages or domains (e.g., product reviews, tweets) will assess the generalizability of the models.
- **Integration with Pre-trained Embeddings:** Using advanced embeddings such as GloVe, FastText, or BERT could further improve the semantic understanding and performance of LSTM models.
- **Exploration of Transformer Architectures:** With the advent of transformer-based models like BERT and RoBERTa, future work can explore comparisons or hybrid integrations with LSTM architectures.

REFERENCES

- [1] Md. Shafiqul Islam et al., “Challenges and future in deep learning for sentiment analysis: a comprehensive review and a proposed novel hybrid approach,” *Artificial Intelligence Review*, vol. 57, no. 3, Feb. 2024, Doi: <https://doi.org/10.1007/s10462-023-10651-9>.
- [2] A. Sârbu, Alexandru Romaniuc, and Anca Gavrilas, “Improving Sentiment Analysis with Neural Networks,” *International conference KNOWLEDGE-BASED ORGANIZATION*, vol. 30, no. 3, pp. 1–6, Jun. 2024, Doi: <https://doi.org/10.2478/kbo-2024-0095>.
- [3] W. Yue and L. Lei, “Sentiment Analysis using a CNN-BiLSTM Deep Model Based on Attention Classification,” *Information*, vol. 26, no. 3, pp. 117–162, Sep. 2023, Doi: <https://doi.org/10.47880/inf2603-02>.
- [4] S. Kalluri, “Deep Learning Based Sentiment Analysis,” *DIVA*, 2023. <https://www.diva-portal.org/smash/record.jsf?dsid=-3248&pid=diva2%3A1741487> (accessed Jan. 31, 2025).
- [5] A. Karim et al., “A Comparative Study of Different Deep Learning Model for Recognition of Handwriting Digits,” *SSRN Electronic Journal*, 2021, Doi: <https://doi.org/10.2139/ssrn.3769231>.
- [6] R. Vatambeti, S. V. Mantena, K. V. D. Kiran, M. Manohar, and C. Manjunath, “Twitter sentiment analysis on online food services based on elephant herd optimization with hybrid deep learning technique,” *Cluster Computing*, Feb. 2023, Doi: <https://doi.org/10.1007/s10586-023-03970-7>.
- [7] S. M. Saqib, T. Naeem, S. Ahmad, and A. S. Alorfi, “Evaluation of Deep Learning Approaches for Sentiment Analysis,” *VAWKUM Transactions on Computer Sciences*, vol. 11, no. 1, pp. 26–41, Mar. 2023, Doi: <https://doi.org/10.21015/vtse.v11i1.1207>.
- [8] A. Benoit, Christophe Escriba, D. Gauchard, A. Esteve, and C. Rossi, “Analyzing and Comparing Deep Learning Models on an ARM 32 Bits Microcontroller for Pre-Impact Fall Detection,” *IEEE Sensors Journal*, vol. 24, no. 7, pp. 11829–11842, Apr. 2024, Doi: <https://doi.org/10.1109/jsen.2024.3364249>.
- [9] H. Deng, Daji Ergu, F. Liu, Y. Cai, and B. Ma, “Text sentiment analysis of fusion model based on attention mechanism,” vol. 199, pp. 741–748, Jan. 2022, Doi: <https://doi.org/10.1016/j.procs.2022.01.092>.
- [10] A. Karim et al., “A Comparative Study of Different Deep Learning Model for Recognition of Handwriting Digits,” *SSRN Electronic Journal*, 2021, Doi: <https://doi.org/10.2139/ssrn.3769231>.

- [11] R. Chandra and B. Vaddatti, "Sentiment Analysis using Deep Belief Network for User Rating Classification," Regular issue, vol. 10, no. 8, pp. 87–91, Jun. 2021, Doi: <https://doi.org/10.35940/ijitee.h9233.0610821>.
- [12] M. Kamyab, G. Liu, and M. Adjeisah, "Attention-Based CNN and Bi-LSTM Model Based on TF-IDF and GloVe Word Embedding for Sentiment Analysis," Applied Sciences, vol. 11, no. 23, p. 11255, Nov. 2021, Doi: <https://doi.org/10.3390/app112311255>.
- [13] N. Ouyang, "Analyze IMDb movies by sentiment and topic analysis," Environment and Social Psychology, vol. 8, no. 3, Oct. 2023, Doi: <https://doi.org/10.54517/esp.v8i3.1958>.
- [14] G. Dubey et al., "A Hybrid Convolutional Network and Long Short-Term Memory (HBCNLS) model for Sentiment Analysis on Movie Reviews," International journal on recent and innovation trends in computing and communication, vol. 11, no. 4, pp. 341–348, May 2023, Doi: <https://doi.org/10.17762/ijritec.v11i4.6458>.
- [15] S. Yin and G. Zhong, "TextGT: A Double-View Graph Transformer on Text for Aspect-Based Sentiment Analysis," Proceedings of the ... AAAI Conference on Artificial Intelligence, vol. 38, no. 17, pp. 19404–19412, Mar. 2024, Doi: <https://doi.org/10.1609/aaai.v38i17.29911>.
- [16] Sri Utami, Kemas Muslim Lhaksana, and Yuliant Sibaroni, "Deep Learning and Imbalance Handling on Movie Review Sentiment Analysis," SinkrOn, vol. 8, no. 3, pp. 1894–1907, Jul. 2023, Doi: <https://doi.org/10.33395/sinkron.v8i3.12770>.
- [17] Z. Yin et al., "DPG-LSTM: An Enhanced LSTM Framework for Sentiment Analysis in Social Media Text Based on Dependency Parsing and GCN," Applied sciences, vol. 13, no. 1, pp. 354–354, Dec. 2022, Doi: <https://doi.org/10.3390/app13010354>.
- [18] K. L. Tan, C. P. Lee, and K. M. Lim, "A Survey of Sentiment Analysis: Approaches, Datasets, and Future Research," Applied Sciences, vol. 13, no. 7, p. 4550, Apr. 2023, Doi: <https://doi.org/10.3390/app13074550>.